

Algoritmo genético para geração de horários escolares

1. Estratégias para uma Inicialização Eficiente

A criação da população inicial é crucial para o desempenho do AG. Embora a geração puramente aleatória garanta diversidade, o uso de **heurísticas** pode iniciar o processo com indivíduos de maior "fitness" (aptidão).

- **Ordenação por Restrições:** Uma técnica eficiente é alocar primeiro as disciplinas com maior número de restrições (ex: horários específicos solicitados por professores ou disciplinas compartilhadas). No PHP, isso pode ser feito ordenando o array de disciplinas antes de iniciar o loop de alocação.
- **Contadores de Conflitos:** Durante a geração de cada indivíduo, pode-se implementar um contador que verifica a quantidade de conflitos em uma coluna antes de alocar. Se houver muitos conflitos, o sistema tenta uma nova posição aleatória (limitada a um número de tentativas) antes de aceitar a alocação, reduzindo o número de choques iniciais.
- **Preenchimento por Diagonais:** Para minimizar colisões iniciais entre turmas e professores, pode-se preencher as matrizes de horários seguindo padrões diagonais, o que ajuda a distribuir a carga horária de forma mais uniforme no espaço de busca inicial.
- **Semeadura (Seeding):** Em PHP, é possível salvar as melhores soluções de execuções anteriores em um banco de dados (como MySQL ou PostgreSQL) e utilizá-las para "semear" parte da população inicial em novas execuções, acelerando a convergência.

2. Representação de Dados e Estrutura em PHP

A escolha da representação do "cromossomo" impacta diretamente a memória e o processamento.

- **Matrizes e Vetores:** Recomenda-se o uso de arrays multidimensionais ou coleções de objetos para representar a grade horária, onde cada elemento (gene) representa uma tupla <professor, disciplina> ou um código identificador.
- **Representação Vetorial:** Codificar o horário de cada professor como uma cadeia de números inteiros simplifica as operações de mutação e cruzamento (crossover).

3. Gerenciamento de Restrições e Complexidade

A aplicação deve ser capaz de distinguir entre restrições rígidas (*hard*) e leves (*soft*).

- **Restrições Rígidas (Hard):** Devem ser resolvidas ou minimizadas na fase inicial, como evitar que um professor ministre duas aulas simultaneamente.
- **Restrições Leves (Soft):** Devem ser tratadas pela função de avaliação para melhorar a qualidade do horário (ex: evitar "janelas" vago no horário do professor ou priorizar o turno da manhã).
- **Disciplinas Compartilhadas:** Para eficiência, disciplinas que ocorrem simultaneamente em diferentes cursos devem ser tratadas como uma unidade

única durante a inicialização para garantir que o mesmo professor e horário sejam mantidos em todas as grades envolvidas.

4. Considerações de Implementação

- **Performance:** Como o problema é NP-Completo, o tempo de processamento cresce exponencialmente com o número de entidades. O PHP 8, com o motor JIT (*Just-In-Time*), pode oferecer a performance necessária para os cálculos matemáticos intensivos das gerações.
- **Flexibilidade:** Ferramentas genéricas são difíceis de criar devido às restrições específicas de cada instituição; por isso, a aplicação PHP deve permitir a configuração modular de pesos e limites para cada restrição.

Em suma, a aplicação de heurísticas informadas por restrições na inicialização da população é a forma mais eficaz de reduzir o número de gerações necessárias, tornando o sistema de timetabling em PHP mais rápido e robusto.

Como implementar a ordenação por restrições no PHP?

A implementação da **ordenação por restrições** no PHP para sistemas de *timetabling* baseia-se em utilizar heurísticas para organizar as disciplinas de modo que as mais difíceis de alocar sejam processadas primeiro durante a inicialização da população. Essa estratégia aumenta a probabilidade de gerar indivíduos factíveis e reduz conflitos iniciais.

Abaixo, detalho como estruturar essa lógica no PHP com base nas fontes:

1. Critérios de Ordenação (Prioridade)

Para implementar a ordenação, você deve primeiro definir o que torna uma disciplina "restrita". De acordo com as fontes, a prioridade deve seguir esta ordem:

1. **Disciplinas Compartilhadas:** Devem ser geradas antes das não compartilhadas, pois precisam coincidir em múltiplos cursos simultaneamente.
2. **Densidade de Restrições:** Disciplinas com maior número de restrições de horários específicos (ex: um professor que só pode dar aula em um único turno ou dia) devem ser priorizadas.
3. **Abrangência:** Disciplinas que envolvem o maior número de cursos ou turmas relacionados.
4. **Carga Horária:** Em alguns modelos, ordena-se por blocos de carga horária (ex: 3h, depois 5h, 6h, etc.) para evitar que o algoritmo entre em loops infinitos ao tentar encaixar aulas grandes no final do processo.

2. Implementação Técnica em PHP

No PHP, a forma mais eficiente de realizar essa ordenação é utilizando a função `usort()` ou `array_multisort()` em um array de objetos ou arrays associativos que representam as disciplinas.

Exemplo de lógica de comparação:

```
// Array de disciplinas vindo do banco de dados

$disciplinas = [

    ['id' => 1, 'compartilhada' => true, 'num_restricoes' => 5, 'num_cursos' => 3],
    ['id' => 2, 'compartilhada' => false, 'num_restricoes' => 2, 'num_cursos' => 1],
    // ... outras disciplinas
];

usort($disciplinas, function($a, $b) {

    // 1. Prioriza disciplinas compartilhadas (true vem antes de false)
    if ($a['compartilhada'] !== $b['compartilhada']) {
        return $b['compartilhada'] <=> $a['compartilhada'];
    }

    // 2. Se ambas forem iguais no critério acima, prioriza quem tem mais restrições
    if ($a['num_restricoes'] !== $b['num_restricoes']) {
        return $b['num_restricoes'] <=> $a['num_restricoes'];
    }

    // 3. Critério de desempate: número de cursos envolvidos
    return $b['num_cursos'] <=> $a['num_cursos'];
});
```

3. Integração com a Inicialização "Greedy"

Após a ordenação, a alocação segue um padrão **Guloso (Greedy)**:

- O algoritmo percorre a lista ordenada e, para cada disciplina, tenta encontrar o primeiro slot livre que satisfaça as restrições rígidas (*hard constraints*).
- Uma vez alocados os itens mais restritos (como as disciplinas compartilhadas), o sistema define o horário e o professor, replicando essa escolha para todas as instâncias daquele compartilhamento antes de seguir para as disciplinas simples.

4. Vantagens Práticas

- **Eficiência Temporal:** Ao ordenar antes de alocar, o sistema evita tentativas aleatórias frustradas, o que é crucial em problemas NP-Completo onde o espaço de busca é imenso.

- **Qualidade Inicial:** A população começa com um *score* de aptidão (*fitness*) muito mais alto, acelerando a convergência do Algoritmo Genético.
- **Redução de Conflitos:** Minimiza a ocorrência de *clashes* (professores em dois lugares ao mesmo tempo) logo na primeira geração.

Quais são as vantagens do motor JIT no PHP 8?

Vantagens do Motor JIT no PHP 8 para Geração de Horários

O motor JIT (*Just-In-Time*) altera a forma como o PHP executa o código, compilando partes do código em instruções de máquina em tempo de execução, em vez de apenas interpretá-las. No contexto do desenvolvimento de um sistema de *timetabling* (que é um problema NP-Completo e computacionalmente caro), as vantagens seriam:

- **Melhoria em Tarefas CPU-Bound:** O JIT oferece ganhos de performance significativos em aplicações que dependem fortemente do processador, como é o caso dos cálculos matemáticos e lógicos intensivos exigidos pelas sucessivas gerações de um Algoritmo Genético.
- **Velocidade em Loops de Evolução:** Algoritmos Genéticos realizam milhares de iterações em loops para avaliar a aptidão (*fitness*) de cada indivíduo da população. O JIT é especialmente eficiente em otimizar esses "hot spots" de código que são executados repetidamente.
- **Processamento de Grandes Estruturas de Dados:** Como a representação de horários geralmente envolve matrizes multidimensionais complexas (genes e cromossomos), o JIT ajuda a acelerar a manipulação dessas estruturas na memória.
- **Eficiência na Função de Aptidão:** A função de avaliação, que precisa verificar diversas restrições (*hard* e *soft*) para cada solução candidata, torna-se consideravelmente mais rápida, permitindo que o algoritmo processe mais gerações em menos tempo ou trabalhe com populações iniciais maiores.

Resumo do contexto: Embora o PHP tradicionalmente se destaque em aplicações web (I/O-bound), a introdução do JIT no PHP 8 torna a linguagem muito mais competitiva para algoritmos de IA e otimização, como os discutidos nas fontes, que anteriormente exigiriam linguagens como C++ ou Java para manter um desempenho aceitável.

Como funciona o preenchimento por diagonais na matriz inicial?

O **preenchimento por diagonais** na matriz inicial é uma técnica de inicialização heurística utilizada para organizar a alocação de turmas e professores de forma a **minimizar colisões (conflitos) logo no início do processo**.

o funcionamento desse método segue esta lógica:

1. Mecanismo de Funcionamento

- **Distribuição Inicial:** O processo começa preenchendo as **diagonais da matriz de horários com números crescentes**, que representam as turmas.
- **Garantia de Atendimento:** Essa estratégia parte do princípio de que cada professor obrigatoriamente ministrará ao menos uma aula para cada turma. Ao alocar as turmas em um padrão diagonal, o algoritmo garante que, inicialmente, não haja dois professores alocados para a mesma turma no mesmo slot de tempo.
- **Complementação:** Após esse preenchimento diagonal estruturado, os espaços restantes da grade são submetidos ao **processo comum de inicialização pseudo-aleatória padrão** para completar a carga horária necessária.

2. Desempenho e Resultados

Embora a intenção seja reduzir o número de colisões, os testes práticos relatados nas fontes mostram resultados específicos:

- **Eficiência Média:** O desempenho em termos de número de gerações necessárias para encontrar uma solução final é **semelhante aos outros métodos** (como a geração puramente aleatória ou com contadores de conflitos).
- **Alta Variabilidade:** A principal característica observada foi um **desvio-padrão muito elevado**.
- **Imprevisibilidade:** Isso significa que, enquanto o método pode gerar uma solução válida em **pouquíssimas gerações** em alguns casos, ele também pode levar um **tempo significativamente maior** para convergir em outros, tornando o tempo de execução menos constante do que o método com contador de conflitos.

Em resumo, o preenchimento por diagonais é uma tentativa de impor uma estrutura organizada à população inicial para "espalhar" as turmas de forma uniforme, mas sua eficácia prática depende fortemente da aleatoriedade subsequente necessária para fechar a grade.